

Creating and using ConfigMaps

```
kubectl create configmap env-config \
--from-literal=ENV=development \
--from-literal=LOG_LEVEL=debug
```

config.properties

```
ENV=development
LOG_LEVEL=debug
```

```
kubectl create configmap env-config --from-file=config.properties
```

configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: env-config
data:
  ENV: "development"
  LOG_LEVEL: "Debug"
```

```
kubectl apply -f configmap.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-env
spec:
  containers:
    - name: web-container
      image: nginx
      env:
        - name: ENV
          valueFrom:
            configMapKeyRef:
              name: env-config
              key: ENV
```

```
- name: LOG_LEVEL
  valueFrom:
    configMapKeyRef:
      name: env-config
      key: LOG_LEVEL
```

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-volume
spec:
  containers:
    - name: web-container
      image: nginx
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config
  volumes:
    - name: config-volume
      configMap:
        name: env-config
```

Creating and managing secrets

```
kubectl create secret generic dbsecret \
--from-literal=username=admin \
--from-literal=password=SecretPassword
```

```
kubectl create secret generic tls-secret \
--from-file=cert=./sever.crt \
--from-file=cert=./sever.key
```

secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
```

username: YWRtaW4=
password: U2VjcmV0UGFzc3dvcmQ=

```
kubectl apply -f secret.yaml
```

```
kubectl get secret db-secret
```

```
kubectl get secret db-secret -o jsonpath="{.data.username}" | base64 --decode
```

```
kubectl get secret db-secret -o yaml > db-secret.yaml
```

```
kubectl apply -f db-secret.yaml
```

```
kubectl delete secret db-secret
```

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-env
spec:
  containers:
  - name: db-container
    image: dbimage
    env:
    - name: DB_USER
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: username
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: password
```

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-volume
spec:
  containers:
  - name: db-container
    image: dbimage
    volumeMounts:
    - name: secret-volume
      mountPath: /etc/secrets
  volumes:
  - name: secret-volume
    secret:
      secretName: db-secret
```

type: Opaque

type: kubernetes.io/service-account-token

type: kubernetes.io/dockerconfigjson

type: kubernetes.io/basic-auth

type: kubernetes.io/ssh-auth

type: kubernetes.io/tls

Environment Variables

```
apiVersion: v1
kind: Pod
metadata:
  name: static-env-pod
spec:
```

```
containers:
- name: app1
  image: alpine
  command: ["sh", "-c", "echo $ENV"]
  env:
  - name: ENV
    value: "production"
```

```
env:
- name: ENV_MODE

valueFrom:
  configMapKeyRef:
    name: env-config
    key: ENV
```

```
envFrom:
- configMapRef:
  name: env-config
```

```
env:
- name: DB_USER
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: username
- name: DB_PASS
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: password
```

```
envFrom:
- secretRef:
  name: db-secret
```

env:

- name: POD_NAME
valueFrom:
fieldRef:
fieldPath: metadata.name

env:

- name: ENV
valueFrom:
configMapKeyRef:
name: env-config
key: ENV

env:

- name: DB_PASSWORD
valueFrom:
secretKeyRef:
name: db-secret
key: password

envFrom:

- configMapRef:
name: env-config

envFrom:

- secretRef:
name: db-secret

volumes:

```
- name: config-vol
  configMap:
    name: app-config
volumeMounts:
- name: config-vol
  mountPath: /etc/config
  readOnly: true
```

```
volumes:
- name: config-vol
  configMap:
    name: app-config
volumeMounts:
- name: config-vol
  mountPath: /etc/config
  readOnly: true
```

```
kubectl exec -it <pod-name> -- sh
```

```
kubectl exec -it <pod-name> -- bash
```

```
printenv
```

```
kubectl exec <pod-name> -- env
```

```
kubectl exec <pod-name> -- env | grep LOG_
```

```
kubectl describe pod <pod-name>
```

```
command: ["sh", "-c", "env && sleep 3600"]
```

```
kubectl logs <pod-name>
```

```
kubectl get pod <pod-name> -o yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: resource-demo
spec:
  containers:
  - name: demo-container
    image: nginx
  resources:
    requests:
      memory: "128Mi"
      cpu: "250m"
    limits:
      memory: "256Mi"
      cpu: "500m"
```

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

